

2400-DS1AL # Algorithms for Data Science

[Kokpit](#) / [Moje kursy](#) / [2400-DS1AL#2025L#277553](#) / [Lecture 2](#) / [Lecture notes](#)

Lecture notes

Credits: dr Krzysztof Fleszar

Lecture 2: Running Time

Basic Definitions

We start by defining the most basic notions a bit more formally.

A **computational problem** is defined by:

- the set of possible inputs, called *instances*
- the correct output(s) for each possible input

The last lecture included an example computational problem:

Search problem

INPUT: A non-decreasing array of integers, i.e., $a[0], a[1], \dots, a[n-1]$, and a number v .

OUTPUT: The smallest x such that $a[x] \geq v$, or n if none exist.

An **algorithm** is a sequence of steps which computes a correct output for each possible input.

The last lecture included two algorithms for the search problem (linear search and binary search). In this course, we are concerned with computational problems that are mathematically precisely stated (not e.g. "given a picture, tell if it is a cat or a dog").

How to Compare Algorithms?

Suppose we have two algorithms solving the same computational problem. Which of them is better? Probably the most important characteristics is the *time complexity*, that is, the amount of time used by the program. Typically, algorithms take more time for larger instances, for example, for the search problem, the time/memory complexity would typically depend on n , the size of the array. Therefore, time complexity is given as a function $f(n)$ of parameter(s) describing the instance. It is also possible that our algorithm has different running times for different instances of size n —we are usually concerned with the worst (pessimistic) case, that is, $f(n)$ is the longest running time on instances of size n . We use this approach for the following reasons:

- It is good to give a guarantee that the program runs in the given time.
- The optimistic case is rarely interesting—the linear search problem runs very quickly if it finds v right away, but this happens rarely.
- The average (or typical) case might be difficult to define.
- The pessimistic case often gives a good approximation on how fast our algorithm typically runs.

As usual, our theory might be a bit too idealistic—in practice, we might be perfectly happy with an algorithm which works slowly (or maybe even incorrectly) in the worst case, but works fast in the real world cases. For example, there are programs called *SAT solvers*, which solve the computational problem called SAT. Many problems in the industry can be rephrased as instances of SAT and then be very efficiently solved by SAT solvers, even though it is widely believed that no efficient algorithm exists solving SAT in the worst case. (It depends on the instance, though—some ciphers used in cryptography could be also rephrased as instances of SAT, but these instances are *hard*.)

Now, what unit should we use to measure the time complexity? We will discuss several options.

1. Real Time

This is a rather bad measure, as it strongly depends on the hardware and software used. We want our theory to be more universal. Also we do not want to get swamped with technical details.

2. Number of Operations Executed by a High-Level Programming Language

...such as C++ or Python. This is a very bad try too, because a simply written code line in such languages might be actually a very complex operation (that's why these programming languages are called *high-level*). For example, consider the two following Python programs:

```
tab = [i for i in range(1000000)]
for i in range(100000):
    tab.pop(0)
```

```
tab = [i for i in range(1000000)]
for i in range(100000):
    tab.pop(len(tab)-1) # same effect as tab.pop(-1) or just tab.pop()
```

We create a list of size 1000000 (1M), and then we delete 100000 (100k) elements of it. The second program (which removes elements from the end) looks a bit more complex than the first one (which removes them from the beginning)—yet it is much faster (a fraction of second, versus about 100 seconds)!

A similar thing would happen with C++ vectors. Why does this happen? Real computers are reasonably well approximated by a theoretical model called the *Random Access Machine*. [See the slides for a picture that makes the following discussion more clear.] In this model, the memory of the computer is represented as a sequence of memory cells (you can think of an unbounded array). Each memory cell can contain an integer (usually from a bounded range), and each cell has an index (address) which is its position in the sequence; thus an index is also an integer that we can store in another cell. The Random Access Machine may execute basic arithmetical operations on the contents of the memory cells (for example, *add the contents of memory cells 0 and 1 and put the result in the memory cell whose index is given in the memory cell 2*). Everything in our computers is also represented in such a way. For example, in C++, a vector v is roughly represented as follows: the consecutive memory cells, starting from the memory cell number k , contain the contents of the vector; there is also a memory cell which contains the address k , and the consecutive cell will contain the number of elements in the vector, n . If the vector v is a global variable, the address of the memory cell containing k will be fixed. To

remove an element from the end of the vector, it is sufficient to decrease n —however, to remove an element from the beginning, we also need to move each remaining element in the vector to the previous memory cell, which will require many operations of our RAM! (Well, we could increase k and decrease n , but C++ and Python do not use this approach for technical/performance reasons, and it would not help anyway if we removed an element from the middle.) Python lists use a similar representation, but a bit more complex—since Python is dynamically typed, it is not sufficient to simply give the number representing each element, Python also needs to store information about the type of each element. (Causing such Python programs to work slower and to use more memory.)

3. Number of RAM Operations

As we have said, the Random Access Machine (RAM) is a theoretical model, so we could try to compute the number of steps executed by it. This corresponds quite closely (that is, is roughly proportional) to the real time, but it lacks the mentioned disadvantages of being hardware dependent. However, this is still a bit too complex—we would have to write a Random Access Machine program for each algorithm in order to compute their exact running times! Furthermore, such RAM code looks very unreadable and it is complicated to analyze its running time (and the analysis cannot be automated in general, as we will learn in the last lecture). Thus, we would like even more abstraction. Thus, on one hand the readability of a high-level programming language, and on the other hand the proportionality to the real time.

4. Number of Dominating Operations

In short, the solution is to consider only a small subset of high-level operations, that we call *dominating operations*, and whose total cost is somehow proportional to the real running time. If a dominating operation is slow, then we treat it as a separate function call for which we analyze the running time separately. The discussion below about dominating operations is for the interested student.

In more detail, our goal is simplicity of the analysis and proportionality to the real time (or just to the total number of RAM operations which implies the latter). To achieve simplicity, we want to focus on high-level programming languages or Pseudocode. But what does proportionality mean? In the classical sense, a running time function $f(n)$ is *proportional* to the actual number $g(n)$ of RAM operations if there is a constant C such that $f(n) \cdot C = g(n)$. Unfortunately, such a definition is too strict as it would force us, in spite of everything, to write and analyze the exact RAM code underlying each high-level operation. However, if we allow the value of C to vary a little bit for each n within a small range, the situation becomes very amenable and convenient as we will see below. Therefore, it's sufficient in algorithmics if a running time function $f(n)$ is *roughly proportional* to the actual number $g(n)$ of RAM operations, that is, if it differs from $g(n)$ by at most a constant factor. Precisely stated, this means that there are constants $C1$ and $C2$ such that $f(n) \cdot C1 \leq g(n) \leq f(n) \cdot C2$. In other words, $g(n)$ belongs to the range $f(n) \cdot [C1, C2]$ (instead of forcing it to be exactly $f(n) \cdot C$ for a fixed C).

A practical solution is to write our algorithm in Pseudocode or in a proper high-level programming language, then to assign a cost to every high-level operation such that the cost reflects the amount of underlying RAM operations, and, finally, to sum up all the costs. Since our goal is to be only roughly proportional to the total number of RAM operations (and, thus, to the real running time), the assigned costs can be very vague and only roughly proportional to the actual RAM operations that underlie the respective high-level operation. In fact, **for most high-level operations the cost can be chosen to be 1**. Thus, we often end up with just counting high-level operations. Only for some special operations or function calls, we would choose a different cost (for example, removing the first element of an array with n elements would cost n units).

However, we don't have to be strict about taking every high-level operation into account. Observe that the loops split the code in a natural way into blocks. Since each block has constant many high-level operations (as the total program size is by definition constant), the most-expensive operation of each block is roughly proportional to the total running time of each block. Thus, it suffices to pick a single most-expensive operation from each block in order to obtain a measure of the total running time that is roughly proportional to the real time.

If we think this idea further, we notice that it is not necessary to consider all the blocks. It often suffices to pick such representative operations from a subset of the blocks. At the end, the only important thing is that total cost of our chosen representative operations, that we call *dominating operations*, captures the magnitude of work the algorithm does. More precisely, the total cost of their executions should be roughly proportional to the actual total number of RAM operations of the whole algorithm, and, consequently, to the real running time.

For example, consider the algorithms binary search and linear search. As the dominating operation, we can choose in both algorithms the comparisons of v with an element of the array a . The dominating operation has cost 1. Thus, we can easily compute that the binary search algorithm will execute at most $\lceil \log(n) \rceil$ (the ceiling of $\log(n)$) of these dominating operations, whereas the linear search algorithm will take at least $n-1$ of these in the worst case. Thus, the running time in the first case is $\lceil \log(n) \rceil$, whereas in the second case it is $n-1$.

Unfortunately, by counting dominating operations (or summing up their costs), we cannot always tell for sure which one of two algorithms is faster or slower when measuring the real time (or the actual number of RAM operations). The reason is that the actual number of RAM operations can differ from the number of dominating operations by an arbitrary factor that is bounded only by some constant (independent of the input size n). Thus, for example, if one algorithm A has $2n$ dominating operations, and the other one, algorithm B, $3n$ dominating operations, then any of the three cases is possible:

- algorithm A is faster (obvious),
- they are equal (if they take the same number of RAM operations, say $6n$, which differs by the constant factors 3 and 2 from the running times of A and B, respectively),

- algorithm B is faster (for instance, if algorithm A takes $6n$ RAM operations, but algorithm B only $3n$).

Therefore, when counting dominating operations, it makes no sense to compare algorithms that differ from each other only by a constant factor as we cannot tell for sure which one is really faster or whether they are equal. However, if we have two such algorithms, then we know that their running times are actually very similar as they differ only...by a constant factor. In such a case, we say that both algorithms have the same *asymptotic* running time, that is, that their running times are *asymptotically* equal.

5. Asymptotics

In practice, we abstract even further, using *asymptotics*. The notion of asymptotics will help us to simplify running time functions (that count dominating operations) and, in consequence, to make it much easier to compare algorithms to each other. Note that this notion is fundamental for the course. So take your time to understand it!

Intuitively—if our algorithm requires, say, $8n^2 + 4n + 125$ operations, we would not care at all about the last two summands, since they are small compared to $8n^2$ for larger n ; also, we usually do not care about the constant factor 8, because this factor is too dependent on the specific computer/theoretical model used. Thus, we just say that the running time of our algorithm is *in the order of* n^2 . (We can also say *of the order of* or *on the order of*). Formally, we use the asymptotic notation (also called the Landau notation or the Big Oh notation). Given two functions $f(n)$ and $g(n)$, we say that:

- $f(n) = O(g(n))$ if there exist constants $C \geq 0$ and n_0 , such that for each $n \geq n_0$ we have $f(n) \leq C \cdot g(n)$;
- $f(n) = \Omega(g(n))$ if $g(n) = O(f(n))$;
- $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Intuitively, $f(n) = O(g(n))$ says " $f(n)$ is in the order of at most $g(n)$ ", $f(n) = \Omega(g(n))$ says " $f(n)$ is in the order of at least $g(n)$ ", $f(n) = \Theta(g(n))$ says " $f(n)$ is exactly of order $g(n)$ ". (Two more notations are sometimes used: $o(g(n))$, "of order smaller than $g(n)$ ", and $\omega(g(n))$, "of order greater than $g(n)$ "—we will not be using them, though; therefore, we do not define them here.)

Note that O is not zero but the letter O (pronounced "big-oh"); Θ and Ω are the Greek letters Theta and Omega.

Typical algorithms could have time complexities of order: $\Theta(nd)$ for $d \geq 0$ (typically d is an integer, but real values are possible too), the greater d , the greater the order; the binary search was $\Theta(\log n)$, which is $O(nd)$ for any $d > 0$, and also $\Omega(1)$. Since $O(1)$ means a function that is bounded by a constant, we sometimes use the notation $nO(1)$ to denote any function in the order of at most some polynomial (without giving the actual degree—only that it is bounded by a constant). Any polynomial is $O(2n)$, $2n$ is $O(3n)$, $3n$ is $O(n!)$, $n!$ is $O(nn)$, etc.

Note that $f(n)=O(g(n))$ and $g(n)=O(h(n))$ implies $f(n)=O(h(n))$. Therefore, we also write $O(g(n))=O(h(n))$. Thus, we can write: $O(\log n)=O(n^2)=nO(1)=O(2n)=O(3n)=O(n!)=O(nn)$. Please also note that in the context of the asymptotic notation, the equality symbol '=' is **not bi-directional**: for instance, $O(\log n)=O(n^2)$ does not imply $O(n^2)=O(\log n)$; in fact, $O(n^2)\neq O(\log n)$.

The usage of the equality symbol '=' might seem a bit weird at first glance since $O(g(n))$ actually denotes a class of functions of order at most $g(n)$. Therefore, some people prefer to write $f(n)\in O(g(n))$ and $O(f(n))\subseteq O(g(n))$. However, other people prefer the equality symbol as it has the advantage that we do not need to distinguish between $\in$ and $\subseteq$. We just have to keep in mind that by writing "=" we do not mean equality in the traditional sense. In this course, we will use the equality symbol.

Memory Complexity

Another most important characteristics is the *memory complexity*—i.e., the number of working memory units used by our algorithm. Most of what we said about time complexity also applies to memory complexity. In particular, we prefer to abstract from the technicalities and just use the asymptotic notation.

Example algorithms and their time and memory complexities will be given in the next lecture.

Ostatnia modyfikacja: wtorek, 2 czerwca 2026, 22:17

[◀ Recording](#)

Przejdź do...

[Lecture 3 & 4 ▶](#)

✉ Skontaktuj się z pomocą techniczną

Jesteś zalogowany(a) jako Ondřej Marvan (Wyloguj)

2400-DS1AL#2025L#277553